

# Datacenter Optimization Project Final Report

James Faber

June 5, 2026

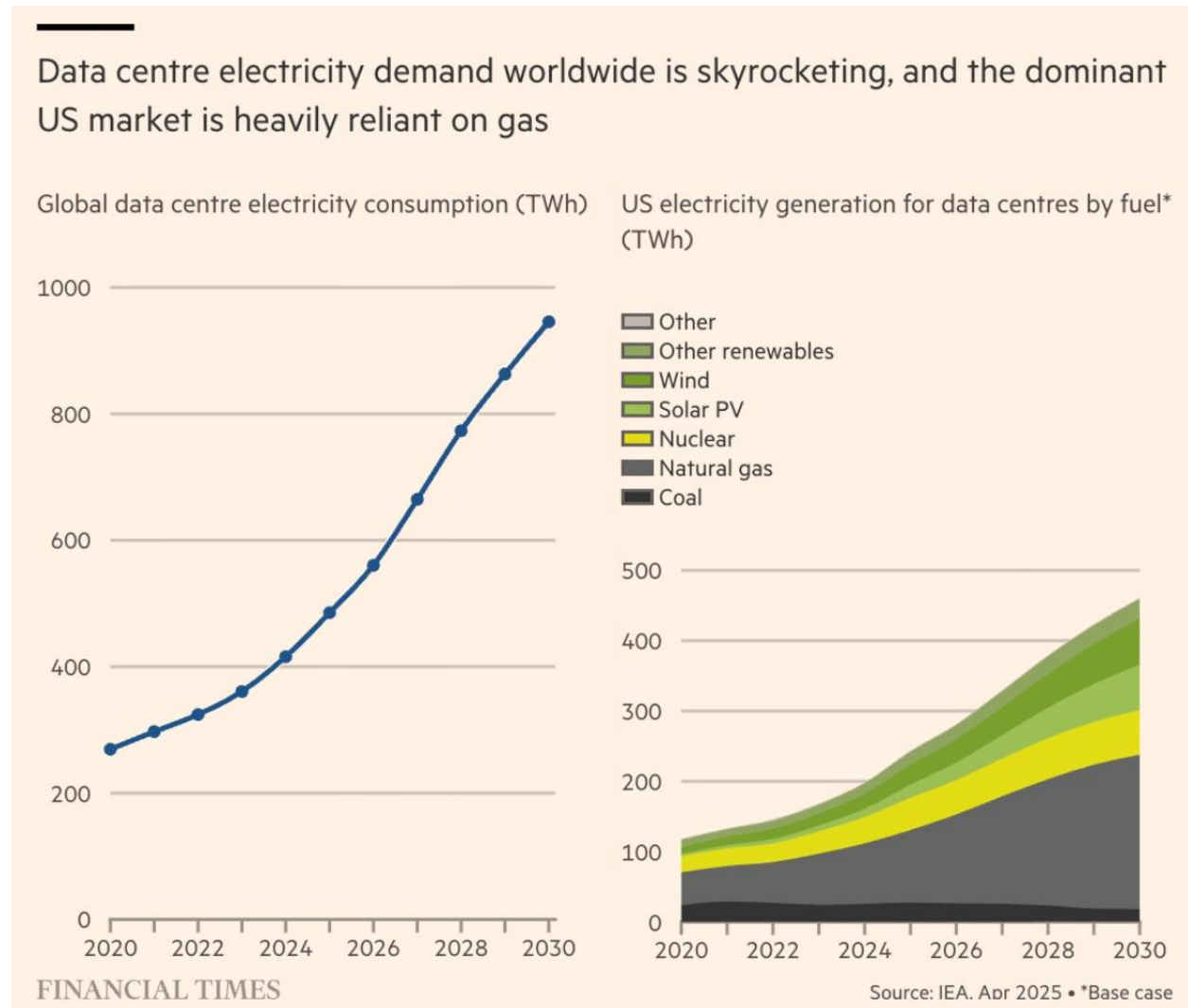
# Table of Contents

|                                     |    |
|-------------------------------------|----|
| Overview .....                      | 3  |
| Project Scope .....                 | 5  |
| Data Used .....                     | 7  |
| Engineering the Final Dataset ..... | 9  |
| Model Training & Results .....      | 10 |
| Model Deployment .....              | 12 |
| Business Impact .....               | 15 |
| Conclusion .....                    | 17 |

## Overview

In 2026, hundreds of datacenters are being built around the world to handle the ever-increasing demand for AI. One of the negative aspects of this build-out that many people will be familiar with is the high electricity consumption that datacenters need to function. This creates problems because the price of electricity goes up where datacenters are built, and generating more power leads to more carbon emissions. But what if we could optimize the operations of datacenters to lower energy consumption instead?

This was the question that I set out to answer at the beginning of this year; and as a practitioner of analytics, I believed that a solution was hiding in the data.



Luckily, there are a few publicly available datasets about datacenter operations. The most detailed dataset by far is the “MIT Supercloud Dataset” released by MIT in 2022, which can be accessed for free via Amazon Web Services (AWS). This dataset recorded individual “jobs” that were run on the MIT datacenter over a 9-month period. A job is just a program that was submitted to the datacenter for its supercomputers to process, and many of the jobs that were run at MIT were AI training models, which makes it especially useful. Despite being four years old at this point, there have not been any comparable datasets published since 2022, and the MIT Supercloud is still actively being used for research purposes.

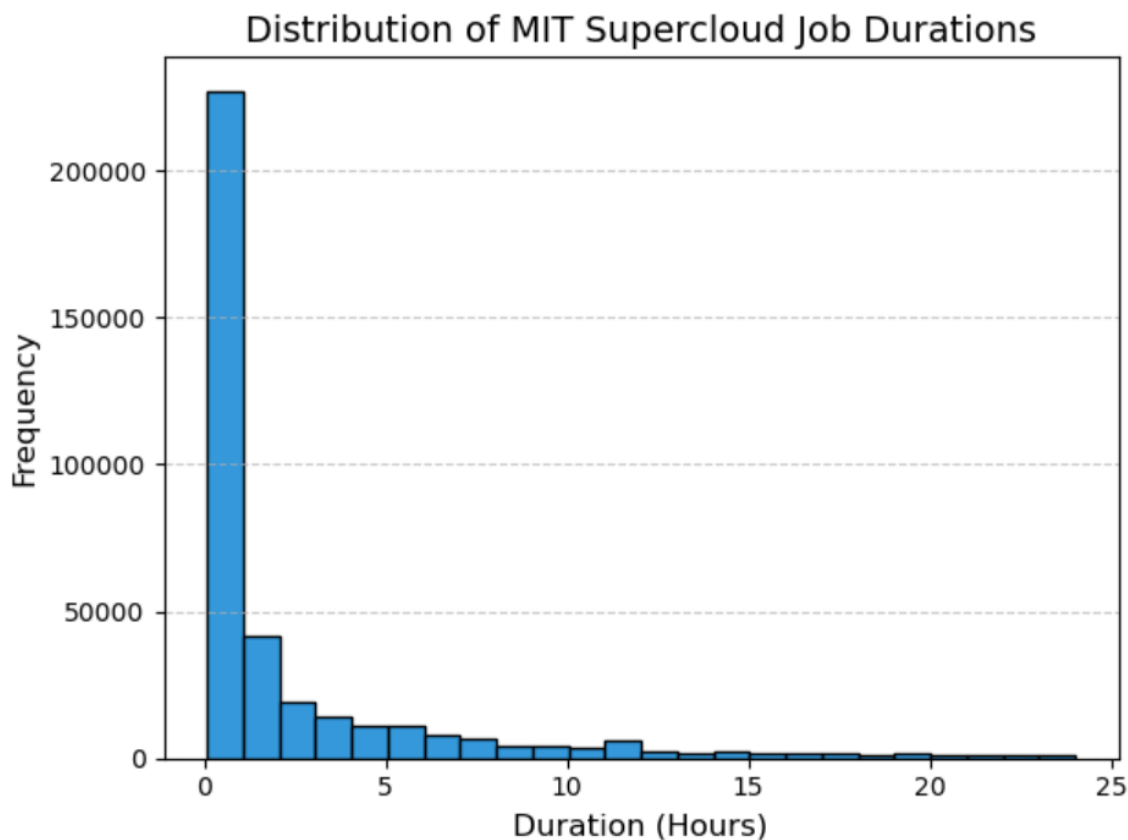
After exploring and analyzing the dataset, I discovered that a whopping 20% of all jobs failed to finish properly or were manually cancelled before they could be completed. Since individual jobs can run for multiple hours or even days at a time, when they don’t finish it is a waste of electricity that could have been used by a “good” job. I will refer to jobs that do not complete successfully as “job failures” going forward for consistency.

This discovery led to my project idea; to build a machine learning model that can predict job failures well before they actually happen. If an accurate prediction can be made, then a job that is very likely to fail could be automatically stopped, which would save hours of electricity usage.

## Project Scope

MIT's dataset is quite large at 2TB in size when fully uncompressed. It has a treasure trove of datacenter monitoring information due to its purpose of supporting academic research. On my GitHub page, I have provided a copy of the original MIT Supercloud whitepaper for reference which goes into detail about all the data that was collected. For this project, I wanted to narrow the scope to something more manageable than the entire repository.

My hypothesis was that a prediction could be made about whether a job would fail by using monitoring information from the early stages of a job. After diving into the available data, I found out that jobs with longer durations had a higher likelihood of failure compared to jobs with shorter durations. I decided to focus my project exclusively on jobs that ran for at least 2 hours, because I assumed 2 hours of data would be sufficient to make a prediction, and there was a large sample size of 15,000 job failures in this subset. As you can see from the chart below, the majority of jobs in the MIT Supercloud dataset ran for less than 2 hours, so my choice to only focus on “over 2-hour jobs” significantly refines the scope.



NOTE: there are also jobs that exceeded 24 hours of runtime, but I chose not to show them on the above graph so that it would be easier to read.

Roughly 70% of jobs in the “over 2-hour” subset completed successfully, so there are plenty of “good” jobs to compare to the jobs that failed. In addition to the 15,000 job failures, the subset also includes 22,000 jobs that were manually cancelled by the user who submitted them. While these cancelled jobs also represent a waste of electricity, I decided that trying to model human behavior was not something I wanted to include in the scope for this project. In other words, I decided to train a predictive machine learning model strictly using job failures and job successes.

Since my model would be performing a binary classification for each job (Success or Failure), the best practice is to train the model using a balanced dataset. What this means is that you don’t want to have a huge amount of successful jobs drowning out the minority of jobs that failed. For my final dataset, I kept the 15,000 jobs that failed- and took a random sample of 15,000 jobs that were completed successfully to get an even 50-50 split. So going forward I’m only concerned with 30,000 total jobs, which is less than 10% of the entire MIT repository. Again, all of the 30,000 jobs in my final dataset had a duration of over 2 hours.

Lastly, I would be remiss if I didn’t mention that in actual fact there is not a single “Job Failure” end-state for a job. There are multiple different ways that a job can fail, including Timeout error, Out-of-Memory error, and General Crash. Each of these failure types are unique, however for this project I lump them together so that my model can predict a binary outcome (Success or Failure). I go into more detail about my reasoning for doing this in the Jupyter Notebooks on my GitHub page.

## Data Used

What kind of datacenter monitoring information could predict a job failure you might ask?

All modern personal computers on the market contain both a Central Processing Unit (CPU) and Graphics Processing Unit (GPU). A CPU handles general tasks and instructions for the system, while a GPU specializes in rapidly processing visuals and graphics.

Datacenters contain thousands of powerful and interconnected CPU's and GPU's that are used to run jobs such as AI training workloads.

For each job in the MIT dataset, there exists a unique log file with CPU timeseries data collected at 10 second intervals. Timeseries data includes information such as the CPU Utilization (how hard the CPU is being worked) by the job, and how much memory it's using. Likewise, for roughly half of jobs there exists a unique GPU timeseries log file with data collected at 0.1 second intervals. GPU's are not allocated for all jobs because they are only used for specialized tasks which require high-throughput arithmetic.

| Step  | Node      | Series | ElapsedTime | EpochTime  | CPUFrequency | CPUTime | CPUUtilization | RSS     | VMSize   | Pages | ReadMB   | WriteMB  |
|-------|-----------|--------|-------------|------------|--------------|---------|----------------|---------|----------|-------|----------|----------|
| batch | r8937440- | 0      | 0           | 1623532271 | 1226         | 0       | 0              | 4748    | 201088   | 0     | 0        | 0        |
| batch | r8937440- | 0      | 10          | 1623532281 | 1226         | 14.36   | 143.6          | 1200880 | 20556980 | 7     | 47.28492 | 8.128551 |
| batch | r8937440- | 0      | 20          | 1623532291 | 1226         | 24.85   | 248.5          | 2654708 | 54341504 | 7     | 252.7403 | 0.127806 |
| batch | r8937440- | 0      | 30          | 1623532301 | 1226         | 23.49   | 234.9          | 5778808 | 57747952 | 7     | 1086.672 | 0.003768 |
| batch | r8937440- | 0      | 40          | 1623532311 | 1226         | 40.59   | 405.9          | 6086164 | 58010096 | 7     | 665.8463 | 0.000639 |
| batch | r8937440- | 0      | 50          | 1623532321 | 1226         | 40.55   | 405.5          | 6229572 | 58042868 | 7     | 663.9005 | 0.000544 |
| batch | r8937440- | 0      | 60          | 1623532331 | 1226         | 40.24   | 402.4          | 6356984 | 58305012 | 7     | 654.0058 | 0.000553 |

The above image is an example CPU Timeseries log showing data from the first minute a job ran. Note that CPU Utilization percentage is inflated by the number of CPU cores allocated to each job.

Now, since each job potentially has thousands of rows of CPU and/or GPU timeseries data, it was not realistic to train my predictive model using the full log files. My idea was to create simple summary statistics about the CPU & GPU data for the first 2 hours that each job was running. For example, one statistic I made was the Median CPU Utilization percentage within the first two hours a job ran. It was my hope that these statistics would make good predictor variables for determining if a job would succeed or fail.

Since I did not inherently know what specific monitoring data could effectively predict a job failure, I ended up creating over 50 new predictor variables (also called "features") that I thought could feasibly do this. Many of these features were the maximum, minimum,

median, and Inter-Quartile Range statistics for data that existed in the CPU/GPU logs. However, I also created some more complex features such as the rate of change (or slope) of a job's memory usage. A full data dictionary can be found on my GitHub page.

## Engineering the Final Dataset

Even though I had refined the project scope to include less than 10% of the MIT Supercloud jobs, I still had a problem. I now needed to process those 50+ features for each one of the 30,000 jobs in my subset, which is a data engineering challenge.

This required finding the CPU and GPU timeseries logs for each job in the AWS S3 bucket the MIT data was stored in, which was not easy because the logs were stored as CSV files within unmarked folders. Cleaning and Processing each log file also took time, as they could often be thousands or tens of thousands of rows each. The computing power needed to accomplish this task was more than my 6-year-old laptop was able to handle.



My answer to this problem was Databricks. Databricks is a cloud-based platform which specializes in processing large volumes of data, fast. It does this by utilizing the Apache Spark engine and seamlessly connecting to other cloud platforms such as AWS. To take advantage of this powerful framework, I converted the SQL code that I wrote to create my features over to PySpark code, and Databricks was able to handle the processing so my computer didn't have to. I was also able to fit all 30,000 jobs within the Databricks free-tier plan by splitting the job list in 4 pieces and running one quarter at a time to get around the usage limits.

Once all the processing had been completed, I ended up with a new 30,000 row table which included metadata about each unique job, and a value for each of the 58 features I engineered (if one existed). Not all jobs used a GPU for example, so variables based on GPU statistics would be blank for those jobs. All of the PySpark code I wrote for this portion of the project is published on my GitHub page and includes comments. The final dataset is also available on GitHub as a CSV file.

## Model Training & Results

Now for the fun part, building a Machine Learning (ML) model to see if my hypothesis is correct- or if I'm just crazy.

To begin, I set aside a random 20% of my final dataset for model validation and testing, using the remaining 80% to train the model. Out of the different types of ML models to choose from, I decided to create a Random Forest model due to its lack of strict data assumptions and resilience to feature correlation.

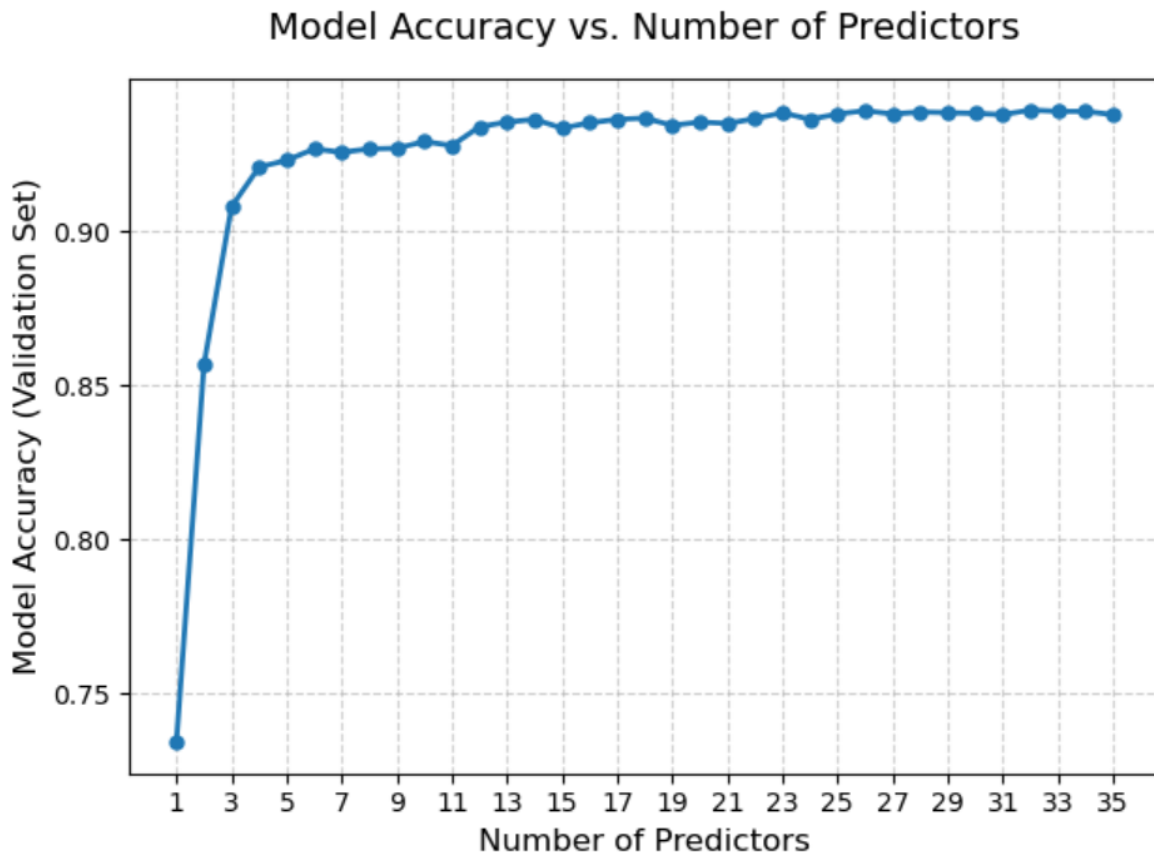
I trained my first model using all 58 predictor variables, and one binary target variable (Job Success or Job Failure). I was astonished to find that this model was able to predict the outcome of the jobs in the validation dataset with a 93% overall accuracy! This was especially encouraging because I used an evenly split dataset which contained 50% successes and 50% failures, so the accuracy number was not inflated by simply guessing "success" every time. Clearly, there are strong signals contained in the time series data for these jobs within the first 2 hours of runtime.

The feature importance scores confirmed that features based on the CPU time series data were by far the most predictive when compared to the GPU-specific features. I decided to train a second model on only jobs which utilized GPU's to be sure, and even that model confirmed that it was the CPU data that determined whether a job would crash. I wasn't even disappointed that all the time I spent engineering those GPU variables was for naught, because an overall accuracy of 93% was a massive result.

However, a random forest model that utilizes 58 predictor variables is complex and difficult to interpret, so I decided to continue testing to see if I could simplify the model without sacrificing accuracy. I removed features that were less predictive than a random variable, and features that were over 90% correlated with another, more important feature. This reduced the total to 35 features, and after re-training the model, it still achieved a 93% accuracy score.

Still wanting to reduce the number of predictors further, I engineered a loop in my python code to train 35 separate models, where the first model would only have one predictor variable and the last model would have all 35. This allowed me to identify a point of diminishing accuracy returns as the number of features is increased. I found that with only

4 features, my model achieved a 92% accuracy score, so I decided to go with this for my final model to have a good balance of accuracy and simplicity.

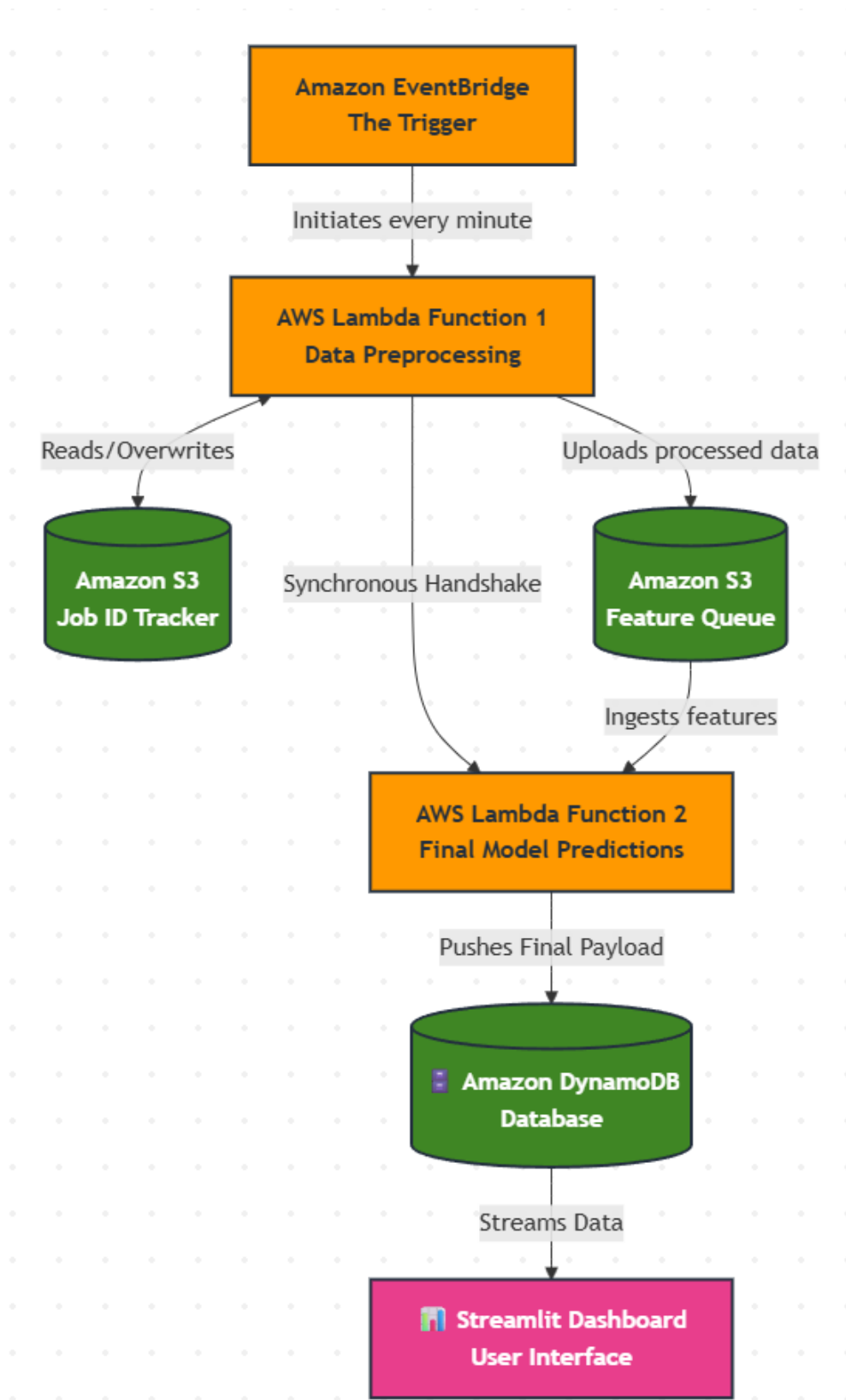


After selecting the final four predictor variables, I did some last-minute model tuning to create the final model. When predicting the outcome of the “test” dataset that had not been seen by any previous model, the final model scored an overall accuracy of 92.4%. While this was not quite as high as the model with all 58 features, it was still an excellent result and much easier to interpret. If you’re curious about which features made the final cut, you can find that information on my GitHub page in the file “Step6\_Modeling\_&\_Analytics”.

## Model Deployment

To simulate a real-world deployment for my final model, I created an AWS workflow that can process raw data directly from the MIT Supercloud S3 bucket, generate a prediction, and push the results to a live web dashboard. In the original MIT dataset, a new job was starting roughly once every minute, so I designed this workflow to be able to select a random job every minute for processing. In a real deployment, I would have set up the workflow to select a job as soon as it hit the 2-hour runtime mark and automatically stop the job if the prediction is a failure.

I chose to deploy my final model to AWS because the source data is stored in AWS, and I was able take advantage of the free-tier allowances for AWS Lambda, S3, ECR, and DynamoDB. Deploying to other platforms such as Databricks would have incurred more charges for being “always on”, so the choice was easy to make. Below is a visualization of how the workflow is currently set up:



I first created my own Simple Storage Service (S3) bucket and uploaded a table containing all 125,000 jobs in the original “over two hour” subset. This table also contained metadata about each job, and a unique link to the MIT S3 folder for each job, which I appended using Databricks. Next I created an AWS Lambda function to select a random job from the table in my S3 bucket and use the MIT S3 link to import the CPU timeseries log for that job. The function then processed the log to calculate the four feature values that were needed for my random forest model and exported this data as a new csv file to a folder in my S3 bucket.

Once the first Lambda function finished running, I set it to automatically trigger a second Lambda function (“Function 2”). Function 2 contained my random forest model and had to be created via a Docker container because it required the scikit-learn library to work. The second function proceeded to ingest the most recent csv file in the “Processed Jobs” folder of my S3 bucket and run the four feature values through the 100 decision trees contained in the random forest model. The two outputs were the prediction (Completed or Failure), as well as the confidence percentage. Confidence in this context is the percentage of decision trees that “voted” for one outcome. The decision threshold was 50%, so if the model had 51% confidence in a job failure, the decision would be job failure, and vice versa for a completed job.

Those two outputs were then placed onto a DynamoDB table, which my live Streamlit dashboard is “pointed” to. When looking at the dashboard, you will notice that 1 prediction is made every hour. This is because the AWS workflow begins to cost a small amount of money when making 1 request per minute for an entire month, and I wanted to stay within the free tier for this project even though the workflow is physically able to handle 1 request per minute. The link to the live dashboard can be found on my GitHub page, and I would recommend viewing it on a desktop because it is not optimized for mobile devices.

## Business Impact

Since we have data about the duration of every job in the MIT Supercloud dataset that failed, it's possible to obtain a very rough estimate of how many total hours of electricity could be saved by prematurely terminating jobs that are likely to crash at the 2-hour mark. With a 92% accurate model, the total number of runtime hours that could have been saved over the 9-month recording period from only jobs that failed is roughly 278,000 hours.

We can also use the model to predict job failures on a random sample of jobs that were manually cancelled by the user to get an idea of their potential failure rate. To do this, I processed a sample of 5,000 cancelled jobs via Databricks and found that they were predicted to fail at a rate of 36%. Extrapolating this percentage to the entire pool of cancelled jobs, and accounting for the fact that they have an average duration of 46 hours, I estimate this will save an extra 306,000 compute hours. If we adjust the estimates from 9-months to 1 year, that brings the grand total to 780,000 compute hours potentially saved over the course of a year for all jobs.

It is difficult to determine an exact number for the average amount of electricity one job would consume in an hour, so I went with a conservative estimate of 0.5 kWh of electricity at a cost of \$0.10 per kWh. After doing the math, this results in 390,000 kWh of electricity saved totaling \$39,000 over the course of a year. This is the equivalent of powering 35 homes for 1 year and saving approximately 110 metric tons of carbon dioxide emissions. Again, these numbers constitute a very rough estimate.

It is worth noting that MIT's datacenter is orders of magnitude smaller than the modern commercial datacenters under construction across the country. While I think it is likely that commercial datacenter operators are actively trying to minimize job crashes more than MIT might be, I would be curious to talk to anyone who has industry knowledge about datacenters so I could learn more about them.

Lastly, I want to point out that it is possible to change the decision threshold of the final model to adjust the prediction precision. For example, if the model required a confidence score of 80% rather than 50% to make a "job failure" prediction, this would reduce the number of false positives dramatically. The tradeoff would be that you would not catch nearly as many job failures and would save less compute hours overall. If I were deploying a model to a production datacenter, it could make sense to start out with a higher threshold

value for predicting job failures. This way you could evaluate the effectiveness of the model for a “pilot period” before adjusting it at a later time.

## Conclusion

I'd like to wrap up this report by saying that this project was surprisingly fun to work on and felt like solving a large/complicated puzzle. It made me dig back in my notes from graduate school and figure out how to apply data science concepts to the real world. I also got a chance to work with industry-leading platforms such as Databricks and AWS for the first time in earnest. It was enjoyable and rewarding to think critically about how I wanted to approach this project and then re-adjust as I learned new information.

I was excited to find patterns in the MIT Supercloud dataset that were able to accurately inform predictions. I had no expectations that this was going to be the case, since at the beginning of my project I had little domain knowledge about datacenters. But my hunch turned out to be well founded, and I look forward to applying the skills I learned from this project to other contemporary problems going forward.